

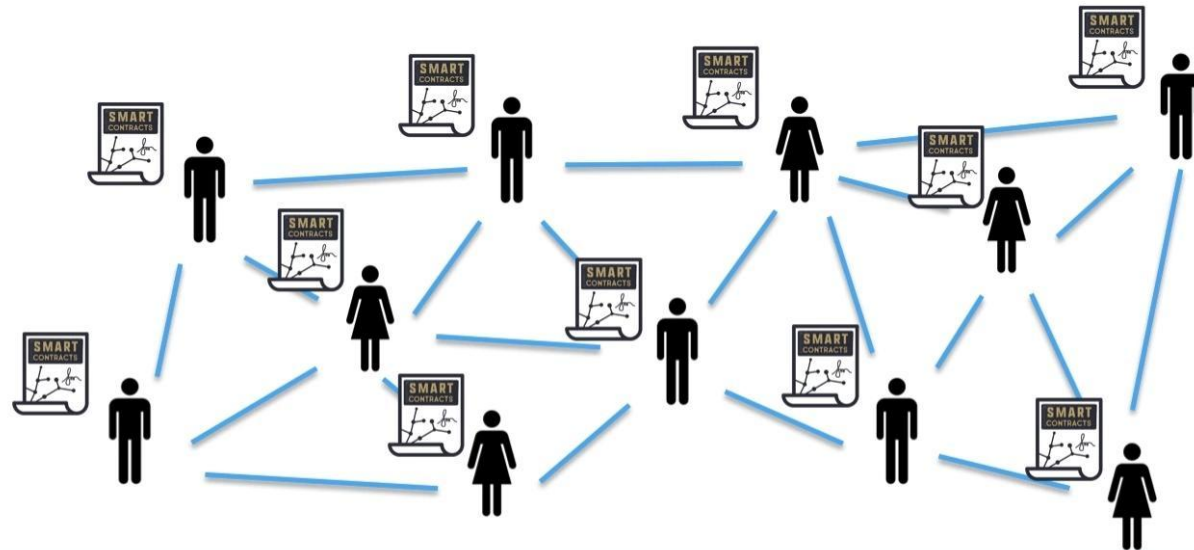
Ethereum

Ethereum Virtual Machine & Gas



Each Node has:

1. History of all smart contracts
2. History of all transactions
3. Current state of all smart contracts



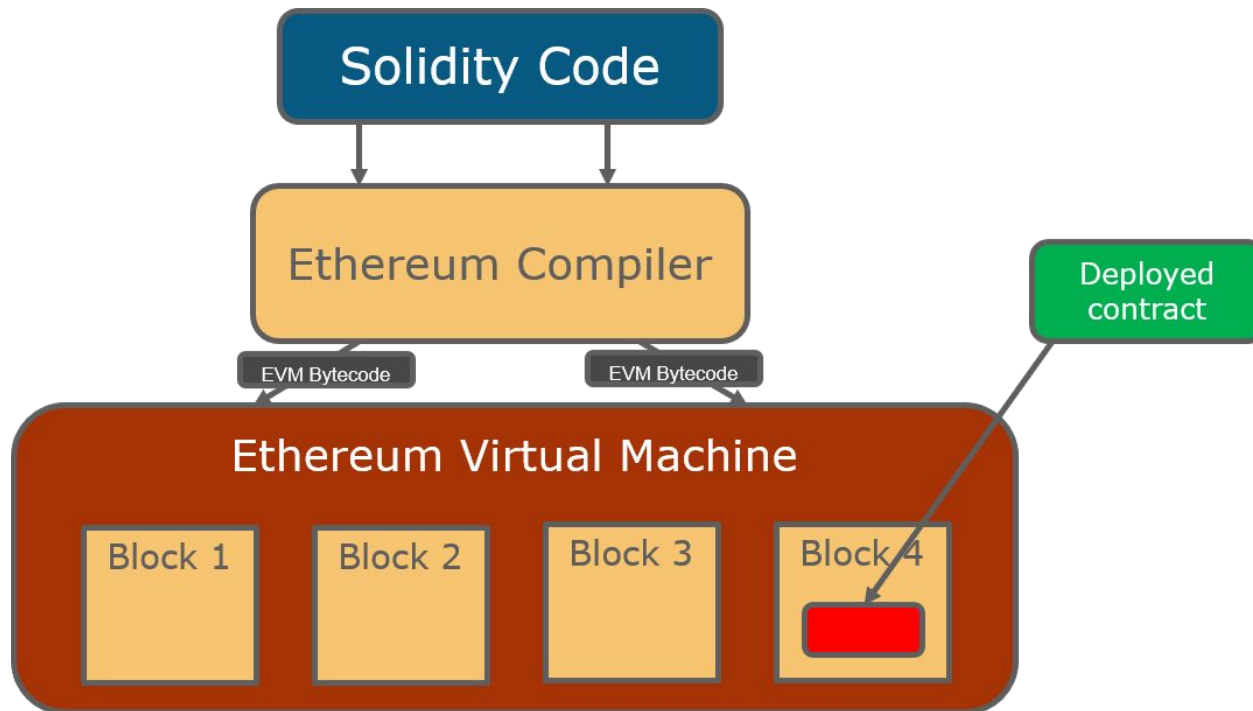
Ethereum Virtual Machine & Gas

Security threats:

- Viruses and access to private files
- Infinite loops

Ethereum Virtual Machine

- Ethereum,- defines a set of generalized protocols
- pillars of the development of decentralized applications.
- At the heart of this, lies the Ethereum Virtual Machine.



Ethereum Virtual Machine is not only completely sandboxed, but also completely isolated. This means that code that is currently running on the EVM has no access to the network or the file-system

The Ethereum Network

- The network is comprised of two types of nodes
 - full nodes
 - light-weight-nodes.

Full nodes

- Contains the entire history of transactions since the genesis block. They are a full-fledged proof of the blockchain network's integrity.
- Full nodes have to contain every transaction that has been verified according to the rules set up by Ethereum's specifications.

The Ethereum Network

Light-weight nodes -contain a subset of the entire blockchain.

- mostly used in e-wallets which have to be light-weight in nature and hence the entire blockchain cannot be stored on them.
- These nodes do not verify every block or transaction and may not have a copy of the current blockchain state. They rely on full nodes to provide them with missing details (or simply lack particular functionality).
- The advantage of light nodes is that they can get up and running much more quickly, can run on more computationally/memory constrained devices, and don't eat up nearly as much storage.

Ether and Gas

- Ether -crypto-currency used to pay for transactions on the ethereum network.
- Ether is also used to buy Gas, which in turn is used to pay for computation within the EVM.
- In the Ethereum ecosystem, Ether (ETH) is the actual cryptocurrency or "money," while Gas is the internal unit used to measure the computational "work" required to perform an action on the network.

Ether and Gas

Value	Mnemonic	Gas Used	Subset	Removed from stack	Added to stack	Notes	Formula	Formula Notes
0x00	STOP		0 zero	0	0	0 Halts execution.		
0x01	ADD		3 verylow	2	1	1 Addition operation.		
0x02	MUL		5 low	2	1	1 Multiplication operation.		
0x03	SUB		3 verylow	2	1	1 Subtraction operation.		
0x04	DIV		5 low	2	1	1 Integer division operation.		
0x05	SDIV		5 low	2	1	1 Signed integer division operation (truncated).		
0x06	MOD		5 low	2	1	1 Modulo remainder operation.		
0x07	SMOD		5 low	2	1	1 Signed modulo remainder operation.		
0x08	ADDMOD		8 mid	3	1	1 Modulo addition operation.		
0x09	MULMOD		8 mid	3	1	1 Modulo multiplication operation.		
0x0a	EXP	FORMULA		2	1	1 Exponential operation.	$(exp == 0) ? 10 :$	If exponent is 0, gas used is 10. If exponent is greater than 0, gas used is 10 plus 10 times a factor related to how large the log of the exponent is
0x0b	SIGNEXTEND		5 low	2	1	1 Extend length of two's complement signed integer.		
0x0c	LT		3 verylow	2	1	1 Less-than comparison.		
0x0d	GT		3 verylow	2	1	1 Greater-than comparison.		
0x0e	SLT		3 verylow	2	1	1 Signed less-than comparison.		
0x0f	SGT		3 verylow	2	1	1 Signed greater-than comparison.		
0x10	EQ		3 verylow	2	1	1 Equality comparison.		
0x11	ISZERO		3 verylow	1	1	1 Simple not operator.		
0x12	AND		3 verylow	2	1	1 Bitwise AND operation.		
0x13	OR		3 verylow	2	1	1 Bitwise OR operation.		
0x14	XOR		3 verylow	2	1	1 Bitwise XOR operation.		
0x15	NOT		3 verylow	1	1	1 Bitwise NOT operation.		
0x16	BYTE		3 verylow	2	1	1 Retrieve single byte from word.		
0x17	SHA3	FORMULA		2	1	1 Compute Keccak-256 hash.	$30 + 6 * (size of input data)$	(size of input data is the paid for the operation plus 6 paid for each word (rounded up) for the input data.
0x18	ADDRESS		2 base	0	1	1 Get address of currently executing account.		
0x19	BALANCE	400		1	1	1 Get balance of the given account.		
0x1a	ORIGIN		2 base	0	1	1 Get execution origination address.		
0x1b	CALLER		2 base	0	1	1 Get caller address.		
0x1c	CALLVALUE		2 base	0	1	1 Get deposited value by the instruction/transaction responsible for this execution.		
0x1d	CALLDATALOAD		3 verylow	1	1	1 Get input data of current environment.		
0x1e	CALLDATASIZE		2 base	0	1	1 Get size of input data in current environment.		
0x1f	CALLDATACOPY	FORMULA		3	0	0 Copy input data in current environment to memory.	$2 + 3 * (number of words copied)$	(number of words copied is the paid for the operation plus 3 for each word copied (rounded up).
0x20	CODESIZE		2 base	0	1	1 Get size of code running in current environment.		
0x21	CODECOPY	FORMULA		3	0	0 Copy code running in current environment to memory.	$2 + 3 * (number of words copied)$	(number of words copied is the paid for the operation plus 3 for each word copied (rounded up).
0x22	GASPRICE		2 base	0	1	1 Get price of gas in current environment.		
0x23	EXTCODESIZE	700	extcode	1	1	1 Get size of an account's code.		
0x24	EXTCODECOPY	FORMULA		4	0	0 Copy an account's code to memory.	$700 + 3 * (number of words copied)$	(number of words copied is the paid for the operation plus 3 for each word copied (rounded up).
0x25	BLOCKHASH	20		1	1	1 Get the hash of one of the 256 most recent complete blocks.		
0x26	COINBASE		2 base	0	1	1 Get the block's beneficiary address.		
0x27	TIMESTAMP		2 base	0	1	1 Get the block's timestamp.		
0x28	NUMBER		2 base	0	1	1 Get the block's number.		
0x29	DIFFICULTY		2 base	0	1	1 Get the block's difficulty.		
0x2a	GASLIMIT		2 base	0	1	1 Get the block's gas limit.		
0x2b	POP		2 base	1	0	0 Remove item from stack.		
0x2c	MLOAD		3 verylow	1	1	1 Load word from memory.		
0x2d	MSTORE		3 verylow	2	0	0 Save word to memory.		
0x2e	MSTORE8		3 verylow	2	0	0 Save byte to memory.		
0x2f	SLOAD	200		1	1	1 Load word from storage.		
0x30	SSTORE	FORMULA		1	1	1 Save word to storage.	$((value != 0) \& \& (20000 \text{ is paid when storage value is set to non-zero from zero. } 5000 \text{ is paid when the storage value's zeroness remains unchanged or is set to zero}))$	
0x31	JUMP		8 mid	1	0	0 Alter the program counter.		
0x32	JUMPI		10 high	2	0	0 Conditionally alter the program counter.		
0x33	PC		2 base	0	1	1 Get the value of the program counter prior to the increment corresponding to this instruction.		
0x34	MSIZE		2 base	0	1	1 Get the size of active memory in bytes.		
0x35	GAS		2 base	0	1	1 Get the amount of available gas, including the corresponding reduction for the cost of this instruction.		
0x36	JUMPDEST		1	0	0	0 Mark a valid destination for jumps.		
0x37	PUSH*		3 verylow	0	1	1 Place * byte item on stack. $0 < * \leq 32$		
0x38	DUP*		3 verylow	*	* + 1	Duplicate *th stack item. $0 < * \leq 16$		
0x39	SWAP*		3 verylow	* + 1	* + 1	Exchange 1st and (* + 1)th stack items.		
0xa0	LOG0	FORMULA		2	0	0 Append log record with no topics.	$375 + 8 * (number of bytes in data)$	(number of bytes in data is the paid for operation plus 8 for each byte in data to be logged.
0xa1	LOG1	FORMULA		3	0	0 Append log record with one topic.	$375 + 8 * (number of bytes in data) + 375$	(number of bytes in data is the paid for operation plus 8 for each byte in data to be logged plus 375 for the 1 topic to be logged.
0xa2	LOG2	FORMULA		4	0	0 Append log record with two topics.	$375 + 8 * (number of bytes in data) + 2 * 375$	(number of bytes in data is the paid for operation plus 8 for each byte in data to be logged plus 2 * 375 for the 2 topics to be logged.
0xa3	LOG3	FORMULA		5	0	0 Append log record with three topics.	$375 + 8 * (number of bytes in data) + 3 * 375$	(number of bytes in data is the paid for operation plus 8 for each byte in data to be logged plus 3 * 375 for the 3 topics to be logged.
0xa4	LOG4	FORMULA		6	0	0 Append log record with four topics.	$375 + 8 * (number of bytes in data) + 4 * 375$	(number of bytes in data is the paid for operation plus 8 for each byte in data to be logged plus 4 * 375 for the 4 topics to be logged.
0xf0	CREATE	32000		3	1	1 Create a new account with associated code.		
0xf1	CALL	FORMULA		7	1	1 Message-call into an account.	Complex -- see yellow paper Appendix H	
0xf2	CALLCODE	FORMULA		7	1	1 Message-call into this account with an alternate account.	Complex -- see yellow paper Appendix H	

Ether and Gas

Feature	Ether (ETH)	Gas
Primary Role	The native currency used for investing, trading, and as a store of value.	A unit of measurement for computational effort (e.g., sending ETH or running a smart contract).
External Value	Market-driven price that fluctuates against fiat currencies (like USD).	No market value outside the Ethereum network; it exists only to measure complexity.
How it's Used	Used to pay the total transaction fee.	Used to calculate the cost of a transaction based on its complexity.

Ether and Gas

How They Work Together

- Think of Ethereum as a car.
- **Gas** represents the amount of fuel needed for a trip (longer trips need more gas)
- **Ether** is the money you use to buy that fuel at the current market price.

Ether and Gas

- The total fee you pay in ETH is calculated as follows:
Total Fee (ETH) = Gas Units Used × Gas Price (in Gwei).
- **Gas Units:** Fixed by the complexity of the operation (e.g., a standard transfer always costs 21,000 gas units).
- **Gas Price:** Measured in **Gwei** (one billionth of an ETH) and fluctuates based on network congestion.
- **Gas Limit:** The maximum amount of gas you are willing to spend on a transaction to ensure it doesn't fail due to insufficient funds.

Ether and Gas

Why Have Both?

- Separating them protects the network from price volatility. If fees were fixed in ETH directly, a sudden spike in the price of Ether would make every transaction on the network instantly more expensive. By using an internal unit (Gas), the cost of the "work" stays constant even if the price of the "currency" (ETH) changes.



Gas Costs from Yellow Paper -- EIP-150 Revision (1e18248 - 2017-04-12)

File Edit View Insert Format Data Tools Add-ons Help



SHARE

Sign in



100%



View only



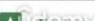
fx

3

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	Value	Mnemonic	Gas Used	Subset	Removed from stack	Added to stack	Notes	Formula	Formula Notes								
2	0x00	STOP	0	zero	0	0	Halts execution.										
3	0x01	ADD	3	verylow	2	1	Addition operation										
4	0x02	MUL	5	low	2	1	Multiplication operation.										
5	0x03	SUB	3	verylow	2	1	Subtraction operation.										
6	0x04	DIV	5	low	2	1	Integer division operation.										
7	0x05	SDIV	5	low	2	1	Signed integer division operation (truncated).										
8	0x06	MOD	5	low	2	1	Modulo remainder operation										
9	0x07	SMOD	5	low	2	1	Signed modulo remainder operation.										
10	0x08	ADDMOD	8	mid	3	1	Modulo addition operation.										
11	0x09	MULMOD	8	mid	3	1	Modulo multiplication operation.										
12	0x0a	EXP	FORMULA		2	1	Exponential operation.	(exp == 0) ? 10 :	If exponent is 0, gas used is 10. If exponent is greater than 0, gas used is 10 plus 10 times a factor related to how large the log of the exponent is.								
13	0x0b	SIGNEXTEND	5	low	2	1	Extend length of two's complement signed integer.										
14	0x10	LT	3	verylow	2	1	Less-than comparison.										
15	0x11	GT	3	verylow	2	1	Greater-than comparison.										
16	0x12	SLT	3	verylow	2	1	Signed less-than comparison.										
17	0x13	SGT	3	verylow	2	1	Signed greater-than comparison.										
18	0x14	EQ	3	verylow	2	1	Equality comparison.										
19	0x15	ISZERO	3	verylow	1	1	Simple not operator.										
20	0x16	AND	3	verylow	2	1	Bitwise AND operation.										
21	0x17	OR	3	verylow	2	1	Bitwise OR operation										
22	0x18	XOR	3	verylow	2	1	Bitwise XOR operation.										
23	0x19	NOT	3	verylow	1	1	Bitwise NOT operation.										
24	0x1a	BYTE	3	verylow	2	1	Retrieve single byte from word										
25	0x20	SHA3	FORMULA		2	1	Compute Keccak-256 hash.	30 + 6 * (size of input data)	30 is the paid for the operation plus 6 paid for each word (rounded up) for the input data.								
26	0x30	ADDRESS	2	base	0	1	Get address of currently executing account.										
27	0x31	BALANCE	400		1	1	Get balance of the given account.										
28	0x32	ORIGIN	2	base	0	1	Get execution origination address.										
29	0x33	CALLER	2	base	0	1	Get caller address.										
30	0x34	CALLVALUE	2	base	0	1	Get deposited value by the instruction/transaction responsible for this execution.										
31	0x35	CALLDATALOAD	3	verylow	1	1	Get input data of current environment.										
32	0x36	CALLDATASIZE	2	base	0	1	Get size of input data in current environment.										
33	0x37	CALLDATACOPY	FORMULA		3	0	Copy input data in current environment to memory.	2 + 3 * (number of words copied)	2 is paid for the operation plus 3 for each word copied (rounded up).								
34	0x38	CODESIZE	2	base	0	1	Get size of code running in current environment.										
35	0x39	CODECOPY	FORMULA		3	0	Copy code running in current environment to memory.	2 + 3 * (number of words copied)	2 is paid for the operation plus 3 for each word copied (rounded up).								
36	0x3a	GASPRICE	2	base	0	1	Get price of gas in current environment.										
37	0x3b	EXTCODESIZE	700	extcode	1	1	Get size of an account's code.										
38	0x3c	EXTCODECOPY	FORMULA		4	0	Copy an account's code to memory.	700 + 3 * (number of words copied)	700 is paid for the operation plus 3 for each word copied (rounded up).								



OPCODE Gas



Ether Denominations

- Ether is the metric unit and has a lot of denominations, which help accurately pay for transactions and gas. The smallest denomination or base unit is called Wei.

For instance, on May 5, 2024, the median transaction fee of the Ethereum blockchain was five Gwei. You could've said, "The gas price was 0.000000005 ETH"

Denominations of Ether		
Unit Name	Wei Value	Number of Wei
Wei (wei)	1 wei	1
Kwei (babbage)	1e3 wei	1,000
Mwei (lovelace)	1e6 wei	1,000,000
Gwei (shannon)	1e9 wei	1,000,000,000
Twei (szabo)	1e12 wei	1,000,000,000,000
Pwei (finney)	1e15 wei	1,000,000,000,000,000
Ether (buterin)	1e18 wei	1,000,000,000,000,000,000

Ether Denominations

- Indian Rupees (INR) use coins/notes for small, discrete amounts (1-1000₹), ETH uses tiny units (Wei, Gwei) for automated smart contracts.

Ether (ETH) Denominations

Ether is highly divisible, allowing for precise, minute transactions on the Ethereum blockchain:

- **1 ETH:** Base unit.
- **1 Finney:** 10^{-3} ETH (0.001 ETH)
- **1 Gwei (Gigawei):** 10^{-9} ETH (0.000000001 ETH) - commonly used for gas fees.
- **1 Wei:** 10^{-18} ETH (0.000000000000000001 ETH) - smallest unit.

Find transaction fee

Q1.

A smart contract execution consumes 85,000 gas units.

Gas price = 40 Gwei. If 1 ETH = \$2500, find the transaction fee in ETH and USD.

$$\text{Total Gas Fee (ETH)} = 85,000 \times 40 \times 10^{-9} = 3.4 \times 10^{-3} \text{ ETH}$$

$$\text{Fee in USD} = 3.4 \times 10^{-3} \times 2500 = \$8.50$$

Q2.Find transaction fee

A Solidity function execution consumes gas units for **Base transaction**: 25,000; **Data input**: (400 bytes × 12 gas each); **Storage update**: 32,000. If the gas price = 45 Gwei, and 1 Gwei = 10^{-9} ETH, find the total gas fee in ETH . (Assume 1 ETH = \$2800.)

Base transaction- minimum fixed gas cost required to execute a transaction .It covers Transaction creation,Transaction validation,Basic execution overhead

```
contract Example {
```

```
    uint public value;
```

always consumes a base cost



```
    function setValue(uint x) public {
```

```
        value = x;
```

```
    }
```

```
}
```

//Base Transaction = "Entry fee" to run any

- **Data input gas** - This is the gas required to send input data to the function.

Example- setValue(123);

Here, the input data includes: Function identifier, Parameters (123)

- Ethereum charges gas per byte of input data.

Given:

- Data size = 400 bytes
Gas per byte = 12

Storage update gas (32,000 gas)

- This is the gas required to **store or modify data in blockchain storage**.

Example - `value = x;` - This updates storage permanently.

Storage operations are expensive because:

- Blockchain must permanently store data
- Every node stores it forever

Typical costs:

- Writing new storage → high gas (like 20,000–32,000)
- Updating existing storage → lower gas

Solution – Q2

• Solution:

$$\begin{aligned}\text{Total gas} &= 25,000 + (400 \times 12) + 32,000 \\ &= 25,000 + 4,800 + 32,000 \\ &= 61,800\end{aligned}$$

Convert gas price to ETH: 45 Gwei = 45×10^{-9} ETH per gas

$$\begin{aligned}\text{Total gas fee in ETH} &= 61,800 \times 45 \times 10^{-9} \\ &= 2.781 \times 10^{-3} \text{ ETH}\end{aligned}$$

$$1 \text{ ETH} = 2800 \text{ USD}$$

$$\begin{aligned}\text{Fee in USD} &= 0.002781 \times 2800 \\ &= 7.79 \text{ USD}\end{aligned}$$

Mining

